

Lab 10

Part A - Polymorphism

1. Create three classes: **EmailNotification**, **SMSNotification** and **PushNotification**.
2. Give all three classes a method called **send()**, but make each method return a different message.
3. Create one object from each class and store them in the same list.
4. Loop through the list and call **send()** on every object.
5. In a comment, explain why the loop does not need to know the exact class of each object.

Part B - Polymorphism with inheritance

1. Create a base class **Document** with a **title** attribute and a method **describe()**.
2. Create **PDFDocument(Document)** and **TextDocument(Document)**.
3. Override **describe()** in both subclasses so they return different descriptions.
4. Create several **PDFDocument** and **TextDocument** objects and store them in one list.
5. Loop through the list and print each document's title and the result of **describe()**.

Part C - Duck typing

1. Create two unrelated classes, for example **Printer** and **Screen**. Do not use inheritance between them.
2. Give both classes a method called **display_status()**.
3. Create objects from both classes and store them in the same list.
4. Loop through the list and call **display_status()** on each object.
5. In a comment, explain why this works even though the classes do not share a base class.

Part D - isinstance()

1. Create a base class **User** and a subclass **AdminUser(User)**.
2. Create an **AdminUser** object.
3. Use **isinstance()** to check whether the object is an **AdminUser**, a **User** and a string.
4. Print all three results.
5. In a comment, explain why the **AdminUser** object is also considered an instance of **User**.

Part E - __str__

1. Create a **Product** class with **name** and **price**.
2. Create one **Product** object and print it before defining **__str__**. Observe the result.
3. Add **__str__** so printing the **Product** gives a useful human-readable description.
4. Create at least three **Product** objects and print them.
5. Use **str()** on one **Product** object, store the result in a variable and print its type.

Part F - __str__ with inheritance

1. Create a base class **Account** with **owner** and **balance**.
2. Add `__str__` to Account.
3. Create **SavingsAccount(Account)** with an additional **interest_rate** attribute. Use **super()** in `__init__`.
4. Override `__str__` in SavingsAccount so its output also includes the interest rate.
5. Create and print both an Account and a SavingsAccount object.

Part G - Inheritance or composition?

1. Create **CPU** with a **model** attribute.
2. Create **Computer** with **brand** and a CPU object. Use composition, not inheritance.
3. Create a CPU object and pass it to a Computer object.
4. Print the computer brand and CPU model through the Computer object.
5. In comments, explain why "Computer HAS-A CPU" makes more sense than "Computer IS-A CPU".
6. For each pair below, write whether you would most likely use **inheritance (IS-A)** or **composition (HAS-A)**: Car / Engine, Manager / Employee, Course / Teacher, Phone / Device.

Part H - Applied challenge: Export system

1. Build a small export system using the concepts from today's lesson.
2. Create a base class **Exporter** with a method **export(data)**.
3. Create at least three subclasses, for example **ConsoleExporter**, **TextExporter** and **SummaryExporter**.
4. Override **export(data)** in every subclass so each handles the same data differently. You do not need to create real files.
5. Add a useful `__str__` method to the exporter classes.
6. Create several exporter objects and store them in one list.
7. Loop through the list and call **export()** on each object to demonstrate polymorphism.
8. Create one additional class that is not part of the Exporter inheritance hierarchy but still provides an **export(data)** method. Show that it can be used by the same calling code.
9. Use **isinstance()** at least once to inspect a meaningful type relationship.
10. Add one example of composition to the program and explain the HAS-A relationship in a comment.